

ppXML: A generic and extensible language for lifecycle modelling of platform products

George Q. Huang^{a,*}, Li Li^a, X. Chen^b

^aDepartment of Industrial and Manufacturing Systems Engineering, The University of Hong Kong, Hong Kong, PR China

^bDepartment of Electromechanical Engineering, Guang Dong University of Technology, PR China

Available online 6 August 2007

Abstract

This paper presents a new language, ppXML (platform product eXtensible Markup Language), for the lifecycle modelling of platform products in agile product development for mass customization (MC). ppXML has multi-folded meanings. Firstly, ppXML provides a set of constructs that are consistent with concepts and methods widely used in platform product development (PPD) for MC. Secondly, derived from XML as a sublanguage, ppXML is a standard and yet extensible modelling language dedicated to the modelling of product variants and platforms reflecting commonality, modularity, scalability and other strategies. Thirdly, ppXML can also be extended for defining web services deployed in the computation grid of web-based decision support systems (DSS) for platform product development and mass customization. Finally, ppXML serves as a standard interface with the product platform repository and a PPD web service registry, together with a set of online facilities for data representation and transformation between different components and parties involved in the web services. ppXML complements the efforts in product lifecycle data modelling by emphasizing the strategic level of product planning using a platform approach.

© 2007 Elsevier B.V. All rights reserved.

Keywords: Product platform; Agile product development; XML; Web services; Mass customization; Manufacturing grid

1. Introduction

Mass customization (MC) embarks a new paradigm for manufacturing industries [1,2]. MC focuses on variety and customization through flexibility and quick responsiveness with the goal of developing, producing, marketing, and delivering affordable products that can satisfy as wide a range of customers as possible [3]. There have appeared a number of approaches to MC. A common element widely advocated by researchers and industrialists is the concept of a product platform [4–6]. A product platform represents basic design features of a family of product variants satisfying a segment of business needs and customer requirements. In its narrowest sense, a product platform consists of common features (e.g. components and assembly relationships) of a family of products. In its broadest sense, a product platform also includes all the knowledge that is required for customizing a specific product within the family according to customer requirements.

Product platforms provide baselines for rapid and accurate customization.

Leading manufacturers have employed the concept of product platforms for their agile product development to enhance their competitiveness. Although isolated, successful cases include Nippondenso panel meters [7], Swatch watches [8], Lutron lighting systems [9], Dell computers [10], Sony Walkmans [11,12], and Philips Medical Systems [13].

Despite benefits reported in numerous industrial cases, the extent and scope to which the platform approach has been practiced in industries have been generally limited. One possible reason for this is due to the high complexity of the product platform and the difficulty of establishing an appropriate platform and then deriving a family of product variants according to specific market and business needs. In order to tackle the problem, a number of major research projects have been initiated and progress has been achieved to some extent.

Majority of the research projects is aimed at delivering a suite of decision support systems (DSS), most of which are centred around product platform development and/or product platform customization. There has been an increasing demand

* Corresponding author. Tel.: +852 28592591; fax: +852 28586535.

E-mail address: gqhuang@hku.hk (G.Q. Huang).

of deploying these DSS as online web services that allow customers to define their requirements, configure their personalized products, place their orders, and track the status of their orders on the Internet. Examples include Dell [14], Cannondale [15], Haworth [16], IDtown [17], Customatix [18], Eyeplanet [19], and Volvo [20].

Most DSS for platform product development (PPD) are data intensive in the sense that a large amount of information and data are required as the inputs in suitable formats and produced as outputs presented in meaningful ways for intended uses. Indeed, extensive knowledge management is involved in PPD decision-making activities. Unfortunately, each research group uses their own formats to represent the contents of information and knowledge, resulting in a situation of inconsistency between various PPD DSS. This situation has hindered the adoption of PPD practice in industries, and has motivated this research. A new language for modelling and representing platform products becomes necessary, despite efforts made in devising new product modelling and definition languages as will be reviewed in the next section.

This paper presents a new modelling and representational language for defining and mass-customizing platform products through emerging web-based decision support systems. Several questions immediately emerge. (1) Why do we need a new model and language for defining platform products across the lifecycle stages? (2) What new constructs should the new model and language provide for platform product lifecycle modeling? (3) How does the new platform product definition language relate to other existing efforts? (4) What roles does the new platform product definition language play in computer aided mass customization?

The remainder of this paper is set out to address the above questions. Section 2 firstly reviews the related literature and identifies the gap for a new platform product definition language (PPDL). Section 3 discusses the requirements and new constructs that the new platform product definition language, ppXML, should meet and provide. Sections 4 and 5 continue the discussion on using the proposed ppXML data structures for defining platform products through a case example. Section 6 sets a wider scene for the research on ppXML and discusses its roles in web-based mass customization environment.

2. Literature review

This section selects and reviews some relevant works mainly for defining products, processes, web services, and product platforms in the literature. The purpose of this literature review is to identify the need for a new language for platform product definition.

2.1. Lifecycle product modelling

Product modelling and representation has been a major focus of research over the past two decades from both communities of product development researchers [21] and information technology (IT) researchers with substantial

investments have been made in the American, European and Asian regions. Generally speaking, PD researchers have been interested in the search of a model for product information and knowledge representation. The model can be used for various design decision support systems involved in different product lifecycles. Early efforts focused on geometrical product information modelling used in computer aided design (CAD) systems and feature-based technological product information modelling used in computer aided process planning (CAPP) systems. Extensions were made both forward for modelling product functions and even product specifications (customer requirements), and backward for further downstream. Several research groups have worked on the well-known Function-Form-Structure-Behaviour Product Model [22,23]. “Design for X” research has further highlighted the important roles of product and process modelling throughout their whole lifecycle [24,25].

Lifecycle product modelling has been a central issue in product data management (PDM) research, still open for further investigation despite all the progresses. According to a major European project entitled PROMISE (<http://www.promise.no/index.php>), a product life cycle can be broadly divided into the following three typical phases: Beginning-of-Life (BOL), Middle-of-Life (MOL), and End-of-Life (EOL). BOL mainly covers product Design and Production, MOL includes Use, Service and Maintenance, and EOL includes several scenarios such as: reuse of the product with refurbishing, reuse of components with disassembly and refurbishing, material reclamation without disassembly, material reclamation with disassembly and, finally, disposal with or without incineration. The PROMISE project sets out to develop a new generation product lifecycle model (PLM) system that will make appropriate product information available through a local wireless connection to product embedded information devices and through remote Internet connection to distributed knowledge repositories.

The substantial efforts made by IT researchers have also evolved from early work on relational data structures for representing geometrical product information to object-oriented data structure for feature-based product modelling. One of the most ambitious efforts was standard for the exchange of product model data (STEP). STEP is only well understood within the STEP researchers and practitioners. It will take a considerable time and effort for ordinary researchers and practitioners involved in product development to become competent in using STEP because of its high level of comprehensiveness and sophistication.

2.2. XML for business data and process modelling

Early product modelling has been focused on the engineering aspect of product information with limited attention to the business and commercial transactional aspect. For example, the Active Product Catalog project was targeted for design practitioners (<http://www.isi.edu/active-catalog/>). Although major e-commerce portals such as IndustryNet and PartNet are dedicated to engineering products and parts, ironically, they

have not been supported by well-structured product model, resulting in limited usability and usefulness.

In contrast, electronic data interchange (EDI) has focused on business and commercial transactional aspect of product (order) information, paying little attention to the engineering aspect. Although still not addressing the engineering aspect, ebXML is a major effort for creating standard and yet extensible modelling language and constructs for business data and processes to support e-business.

Similar efforts are initiated in specific product/industry sectors. For example, aecXML is an XML-based language used to represent information and to facilitate communication of information between the various constituents involved in the architecture, engineering and construction (AEC) including: architects, engineers, contractors, owner/operators, estimators, consultants, materials suppliers, building product manufacturers, and others. This information may be resources such as projects, documents, materials, parts, organizations, professionals or activities such as proposals, design, estimating, scheduling and construction. It is intended to be used as an XML namespace and to facilitate information exchange of AEC data on the Internet (<http://www.iai-na.org/aecxml/mission.php>).

2.3. Product family and platform modelling

Substantial efforts have been made for investigating into the representation of product structures and product platform structures. Naturally, researchers have investigated if it is possible to extend the recent developments in product modelling for the purpose of modelling product platforms and families. For example, the use of STEP in product platform modelling has attracted some attention. STEPml is a library of XML specifications—document type definitions (DTDs) and/or XML schemas—for product data (<http://www.stepml.org/index.html>). However, Mannisto [26] carried out studies and demonstrated fundamental mismatches between STEP constructs and needs of product family modelling.

Further extensions must be sought. Several issues have been recognized for product family and platform modelling. They include the composition, configuration (connecting relationships), attributes and their values (characteristics), and constraints that relate all these factors. Options of all these factors make their modelling and representation very complicated.

One of the common ways is to represent the product structure in terms of hierarchical trees with nodes representing constituent components and edges representing the “has a” relationships between the connecting components. Such a Bill-of-Materials (BOM)-like tree is good for representing the composition (but not configuration) of individual products. However, great challenges exist for using the similar method for representing the composition of product platform. Extensions have been made in several directions. It is important to point out that these recent attempts have all quoted early work by Erens and Verhulst [27] on modelling product family architectures and the work by van Veen [28] on modelling product structures by generic bills-of-material.

One of the common methods of representing the configuration relationships between constituent components of a product is by means of graphs. A graph includes a set of nodes and a set of edges that connect the nodes. Although not originally motivated for modelling product configurations, several XML derived graph description languages have emerged for some other purposes. Examples include Graph eXchange Language (GXL), and eXtensible Graph Markup and Modeling Language (XGMML). The purpose of XGMML is to make possible the exchange of graphs between different authoring and browsing tools for graphs. In particular, GXL was developed to enable interoperability between software reengineering tools and components ([29]; <http://www.gu-pro.de/GXL/>). There are two innovative features in GXL that make it well-suited to an exchange format for software data. Firstly, the conceptual data model is a typed, attributed, directed graph. Secondly, it can be used to represent instance data as well as schemas for describing the structure of the data. The structure of graphs exchanged by GXL streams is given by a schema represented as a Universal Markup Language (UML) class diagram.

Substantial research has been conducted by a strong team of Finnish researchers to develop scheme and method for modelling configurable product families [30]. One of the important aspects of this Finnish research is the representation and modelling of the product configuration knowledge using weight constraint rules [30].

In addition, researchers at the HKUST have been working on a new knowledge intensive model for representing product family architectures with the support of a HKRGC grant and from local industries [31]. Jiao and Tseng [32] set out an initial framework for modelling product family architectures. Further development has been achieved in Du [33].

Finally, Architectural Description Languages have been developed based on eXtensible Markup Language (XML) to represent the software product platforms [34]. Experience and insights learned from researches into both software and manufactured architectures for product families should be cross-examined to achieve cross fertilization for manufactured products as well.

3. ppXML as a platform product lifecycle definition language

ppXML is a new language specially derived from XML for modelling platform products. Its main purpose is to provide common data structures for both the PPD/MC service providers and consumers to define platform-based products for various lifecycle applications. A ppXML document describes a platform product development project completely in a machine operable and user-understandable manner. Section 4 will use a case example to illustrate the detail of a typical ppXML document. This XML-based language is formally defined by the XML Schema Language, named ppXML Schema. ppXML Schema defines the grammars and structures of ppXML language, including which elements and attributes can be used, how they are ordered and nested, and what data types are

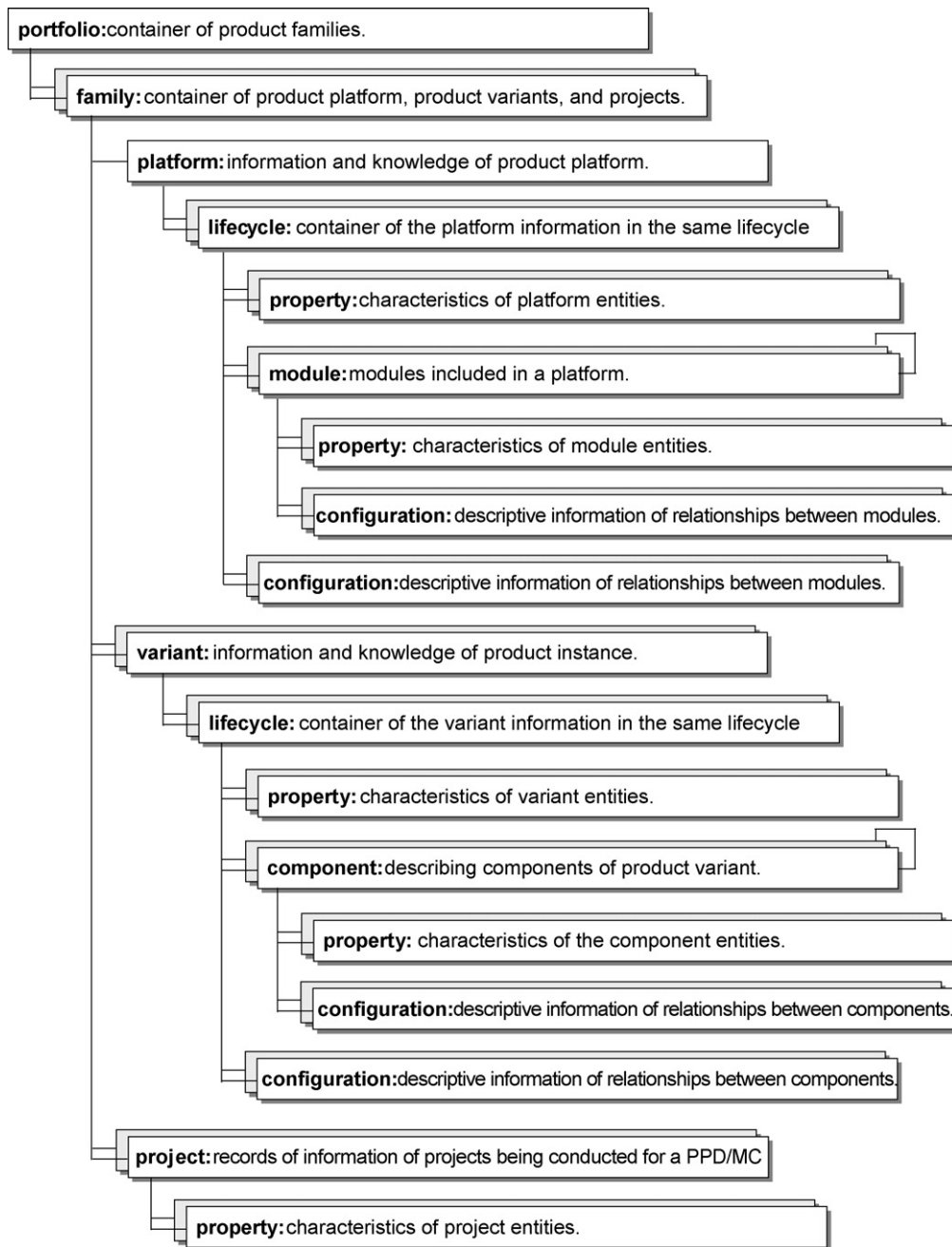


Fig. 1. ppXML data structures and main elements.

permitted in a ppXML document. ppXML Schema is used by the XML parser for verifying ppXML documents to keep the consistency of the platform product information when they are loaded for different application purposes. The content of it can be accessed through the hyperlink: <http://www.digiprise.org/ppXML/ppXML.xsd>. Fig. 1 offers a concise representation of the ppXML language. More detailed discussion of the major ppXML elements is given below.

3.1. Overview of ppXML constructs

ppXML is intended as a definition language for enterprises to define their portfolios of products and product development

projects with a platform approach. Therefore, the root element defined is the **portfolio** entity. The **portfolio** element contains only one type of the **family** element which defines product families or models of a manufacturing enterprise. The **family** element is also a container, including three types of child elements. They are **platform**, **variant** and **project** elements. As each product family consists of several product variants, the **family** element may contain multiple **variant** elements. A **variant** element defines a specific product instance (variant). All product variants within the same product family are derived from the same product platform. For this reason, the **family** element contains one and only one **platform** element. In other words, different product platforms result in correspondingly

different families of product variants. The **project** elements are used for defining product platform and product variant development project information respectively. Each **project** element defines one product development project, uniquely identified by its *id* and *name* attributes. Therefore, **platform**, **variant**, and **project** are the most important constructs provided in ppXML. They are further defined by attributes and other child elements. When they are defined within the family element, their *family* attribute implicitly inherit the *id* value of their container **family** element.

3.2. ppXML constructs for defining product variants

The modelling and definition of product variants are more straightforward than product platforms. Therefore, the discussion will be omitted here. ppXML provides the **variant** element for defining product variants within a product family. The ppXML **component** element is provided for defining the composition of a product variant, appearing within the variant element. The hierarchical nature of the bill of materials of the product variant or instance (IBOM) is implicitly defined through the recursive nesting of the **component** elements. The relationships between components are defined by the **configuration** element in ppXML. Important characteristics of a component element are detailed by **property** elements.

3.3. ppXML constructs for defining product platforms

The modelling and definition of product platforms in ppXML are much more complicated than that of product variants. In the platform, the information what may compose a variant, and the relationships between the compositions are to be expresses. ppXML provides the **platform** element for defining a product platform. Platform definition in ppXML is based on the concept of generic bill of materials. Given the platform, what kind of variants and how many of them can be derived from the platform are determined by the strategies used in the platform. The platform strategies include commonality, modularity, postponement, and scalability. It is important to note how four platform strategies are described in ppXML.

The composition of a product platform is defined by a collection of nested **module** elements. The **module** element defined for a **platform** is comparable to the **component** element of a product **variant**, with additional complexity. Due to the fact that many products have a hierarchical structure, modules are composed of other lower level modules. Such hierarchical nature of the platform's GBOM is reflected by the nesting relationships between **module** elements. There are five types of a **module** element in ppXML as defined by its *type* attribute.

Two important types of modules are “AND”, and “OR”, also noted as “inclusive” and “exclusive” modules. They determine the rules of generating a variant from the platform. An “AND” module represents a compound module that is composed of several sub-modules. All of the sub-modules of an “AND” module must be included in all the customized variants of the family. In this sense, an “AND” module is considered as

a sub-assembly of the platform. An “OR” module is usually an abstract concept. It is not considered as a sub-assembly of all its sub-modules. Instead, only one of its sub-modules nested in this “OR” module is instantiated during the customization process. As a result, only that sub-module is included in the customized product variant. All **module** elements defined within an “OR” module are often called **module options**, module variants, module instances. Which sub-module should be selected depends on customer requirements for the variant product design.

The other three types of modules are bottom level modules, namely, “SCALABLE”, “INSTANCE”, and “EMPTY”. The bottom-level modules are those without any sub **module** elements. If all the characteristics of a module cannot be changed during the customization process, this module is classified into “INSTANCE” module. Otherwise, the module is changeable or scalable in the sense that some of its characteristics can be modified according to the needs. If the function of the “OR” module is optional, then an “EMPTY” module is necessary to be as the child element of this “OR” module element.

These five types of modules on the one hand represent the composition of the platform. On the other hand, they together reflect the four platform strategies. The “OR” type of modules basically represent the platform modularity while the nesting of “AND” and “OR” type of modules determine the platform commonality. For a bottom level module, if all of its parents are “AND” modules, this module is common throughout the whole family of product variants. If there is “OR” parent modules, this module will be only common among the variants composed of this module. Of course, if the bottom level module is an “INSTANCE” module, then its characteristics will be common among the variants with this module. The platform postponement strategy is described by the relative combination of “AND” and “OR” modules with “OR” module elements located near the lower levels of nesting. As for the platform scalability, it is determined by the “SCALABLE” modules and is defined by the **property** elements, which is portrayed as follows.

3.4. ppXML constructs for defining platform product characteristics

The **property** elements provided in ppXML are used for defining the characteristics or parameters of platform products (platforms and/or variants) and their constituent components and modules (and their options). Each **property** element is defined through a number of attributes, including the **id**, **name**, **type**, **lower** and **upper value** for continuous type of properties, **units** of the values, and alternative values for discrete type of properties, etc. The **property** elements are special in several ways. Firstly, they are normally included in defining other entities such as **platform** and **variant**, **module** and **component**, and **project** elements.

Secondly, some property elements of a product platform have fixed values and some have changeable values. Values of fixed properties of a platform are common to all the variants and

cannot be customized. When a property is common, its fixed value is given in the *value* attribute of the **property** element. The value of a semi-variable property of a platform can be selected from a set of given values or series of values during the platform customization process. The series of values are defined in the **option** elements. The value of a fully variable property can be optimized in a relatively free range with little constraints. The *ubound* and *lbound* attributes of the **property** elements tell the range of such kind of variable characteristics.

Thirdly, for customizable property elements, their values can be selected from a continuous or discrete domain. A discrete domain is defined by one or multiple **option** elements bracketed within a property element. A continuous domain is defined by a *ubound* attribute and a *lbound* attribute, and possibly a step by which the value changes within the domain.

Finally, the *lifecycle* attribute can explicitly appear within the property definition. If *lifecycle* = “basic”, the corresponding property is a basic characteristic shared by all lifecycles. If *lifecycle* = “assembly”, this property describes a product characteristic associated with its assembly process. There are two ways of defining lifecycle viewpoints for product platform and variants. One is to specify the value of the *lifecycle* attribute of the product **property** elements. The other way is to define **lifecycle** elements explicitly within the product **platform** and/or **variant** elements to group all the **property** elements. In this latter case, the *lifecycle* attribute of the product **property** elements is given the value of the *viewpoint* attribute of the **lifecycle** element. When **property** elements are defined within a **lifecycle** element, their lifecycle attributes will automatically be assigned to the value of the **lifecycle** element.

3.5. ppXML constructs for defining product lifecycle viewpoints

Product variants of the same product family share the same lifecycle phases. In the literature of product data modelling, the

term viewpoint is often used to group all the product data related to a particular aspect or lifecycle stage in the product development process. The **lifecycle** element is provided in ppXML as a sub-element of either the **platform** element or the **variant** element for defining the stages of the product lifecycle. For example, the product is defined by customer requirements when the lifecycle viewpoint is at the stage of “Customer Requirement Analysis”. Likewise, the product is defined in terms of components and their configuration relationships if the viewpoint is “Embodiment Design”. Manufacturing processes and operations are defined for the product at the stages of “Process Planning” and “Production Planning”.

3.6. ppXML constructs for defining product development projects

From the perspective of product lifecycle, a product development project is concerned with the establishment of the characteristics as defined by **property** elements of a product at one or more **lifecycle** viewpoints as its output. Fig. 2 shows the overview of the product platform development projects from several lifecycle overviews. This process may take characteristics from one or more lifecycle viewpoints as its inputs. By the platform approach, product development is the process of instantiating or customizing the platform into suitable variant(s). For example, the project of “Conceptual product design” develops a platform with its details such as composition, configuration and properties with a set of given customer requirements as the inputs from the lifecycle viewpoint of “Functional”.

Based on the above observations, ppXML provides the **project** element for defining platform product development projects. The input and output lifecycle viewpoints are specified by the **property** elements with the names input and output respectively. Their values correspond to the lifecycle viewpoints defined in by the **lifecycle** elements for the family. In the context of web services and mass customization grid, a remote

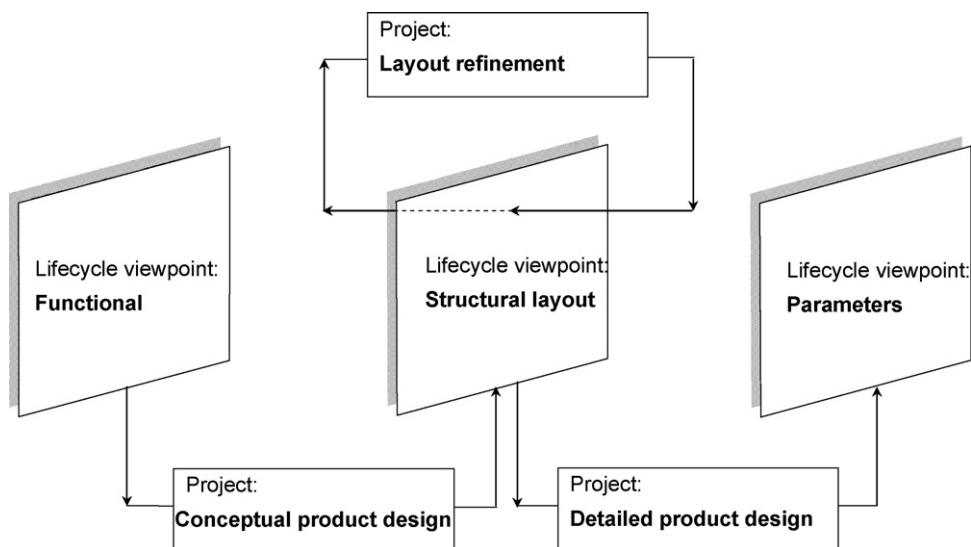


Fig. 2. Project overview of product platform from lifecycle viewpoints.

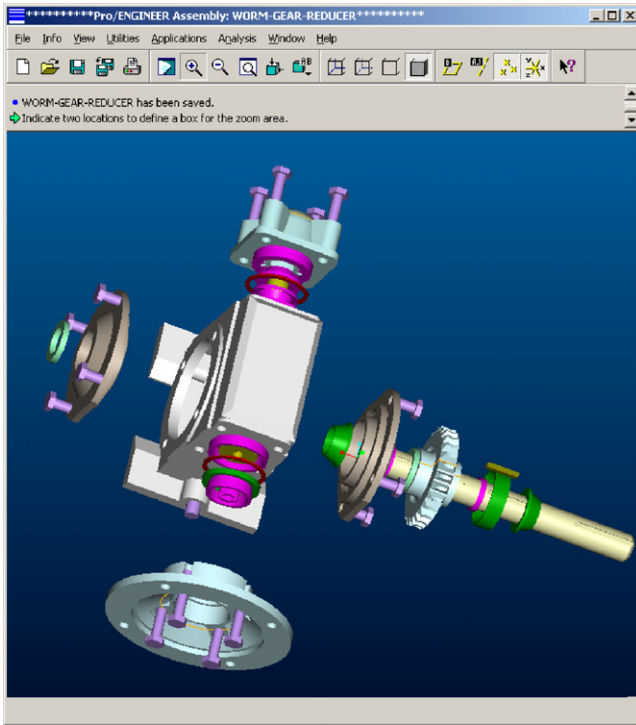


Fig. 3. Exploded assembly of a worm reducer.

decision system is specified for a project through its **property** sub-element with “*id = method*”.

4. ppXML documents for defining platform products

A ppXML document is an XML document, a well formed textual data object. It consists of ppXML entities for defining platform products with the constructs discussed in the previous section. This section shows the use of ppXML to model a typical platform product – transmission boxes shown in Fig. 3. The ppXML document for this platform reducer can be accessed through the hyperlike: <http://www.digiprise.org/ppXML/reducer.xml>. The first few lines are for general declaration purposes such as specifying the XML version, encoding and the underlying XML schema.

The root element **portfolio** (*id = “portfolio1”* and *name = Reducer portfolio*) contains the reducer families and corresponding projects. According to this ppXML document, this portfolio includes two families of transmission boxes, as defined by the two **family** elements. They are gear reducers and worm-gear reducers. This paper uses the second family for the rest of discussion. This “Worm-Gear Reducer” family includes only one **platform** and several **variant** ppXML elements. They will be explained in detail below.

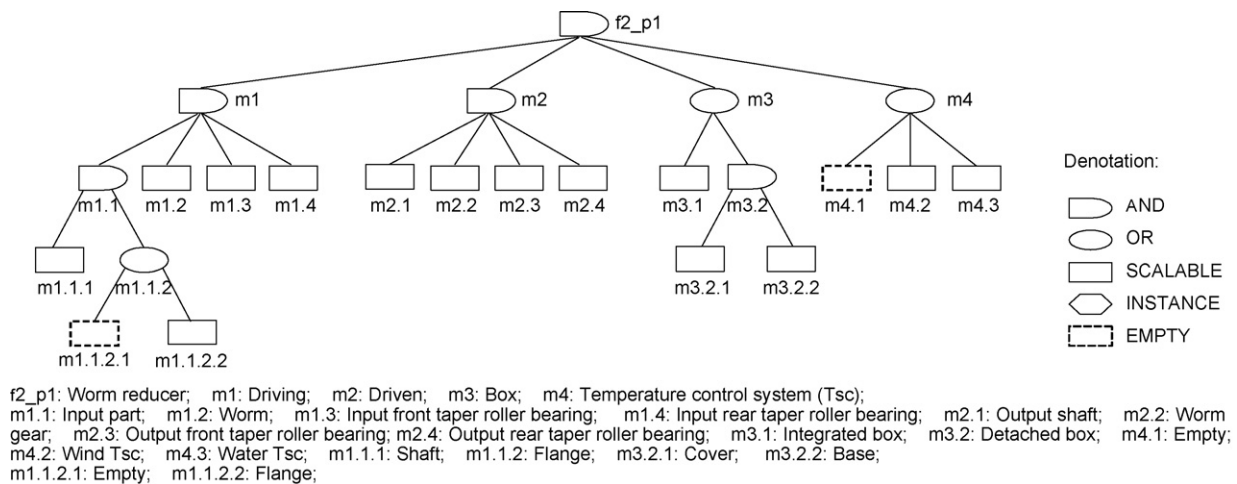


Fig. 4. Composition of worm reducer platform.

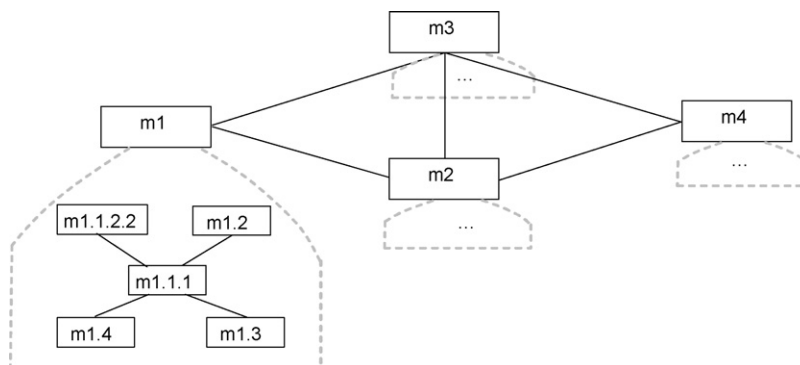


Fig. 5. Configuration of worm reducer platform.

4.1. ppXML platform element/document

The composition, configuration and characteristics of the transmission box (worm reducer) platform are shown schematically in Figs. 4 and 5 and Table 1, respectively.

The composition of the reducer platform is represented as a hierarchical tree (generic bills of materials) shown in Fig. 4. Such hierarchical relationships are naturally and implicitly implemented by the nesting of **module** elements in ppXML. For example, the reducer platform consists of four top-level modules, being modelling as the four **module** elements within the **platform** element. Among these four modules, the driving subsystem (m1) and driven subsystem (m2) are “AND” modules. They in turn consist of lower level modules. For example, the driving subsystem includes input part (m1.1), worm gear (m1.2), front support (m1.3) and rear support (m1.4). The inclusive relationships between the modules are directly captured by the XML nesting structure.

The box (the housing of the worm reducer—m3) and the temperature control system (m4) are “OR” modules, which are realized by their sub-modules being nested in them. These two

“OR” modules, together with other “OR” modules reflect the modularity of the worm reducer platform. This is one of the unique features of the GBOM tree. The design must specify one and only one option when designing a product variant from this platform. That is to say one and only one of its branches can be configured into the IBOM of a reducer. For example, m4 can only be realized by one of its three module options, i.e. m4.1, m4.2, and m4.3 respectively when a worm-gear reducer is generated from this platform.

As for the bottom level modules in the worm reducer platform, most of them are “SCALABLE” modules. For example, modules input shaft (m1.1.1) and output shaft (m2.1) are two bottom-level modules. This tells the platform provides a high scalability. There is no “INSTANCE” module in this platform, while only one “EMPTY” module (m4) implying that the worm reducer may not have the function of temperature controlling. In all, this platform provides the customization space for worm reducers design, resulting in rapid reaction to different customer requirements.

Fig. 5 shows how modules of the reducer platform are related to each other. Such relationships are defined by the

Table 1
Characteristics of worm reducer platform

Module		Property		
id	Name	id	Name	Value
f2_p1	Worm reducer	f2_p1_c1	Ratio	[10,50]
m1.1.1	Input shaft	m1.1.1_c1	Material	45steel
		m1.1.1_c2	Min diameter	[20,200] mm
m1.1.2.2	Flange	m1.1.2.2_c1	Material	45steel
		m1.1.2.2_c2	Type	{big, small}
m1.2	Worm	m1.2_c1	Modulus	[1,25] mm
		m1.2_c2	Teeth number	{1, 2, 4, 6}
		m1.2_c3	Middle diameter	[20,400] mm
		m1.2_c4	Width	{15, 20, 25}mm
m1.3	Input front taper roller bearing	m1.3_c1	Type	[30203,30220]
m1.4	Input rear taper roller bearing	m1.4_c1	Type	[30203,30220]
m2.1	Output shaft	m2.1_c1	Material	45 steel
		m2.1_c2	Output diameter	[20,100] mm
m2.2	Worm gear	m2.2_c1	Modulus	[1,10] mm
		m2.2_c2	Teeth number	[10,100] mm
		m2.2_c3	Middle diameter	[50,500] mm
m2.3	Output front taper roller bearing	m2.3_c1	Type	[30203,30220]
m2.4	Output rear taper roller bearing	m2.4_c1	Type	[30203,30220]
m3	Box	m3_c1	Input shaft height	[100,300] mm
		m3_c2	Output height	[10 0, 400] mm
		m3_c3	Input relation	{R, L, V, W}
m3.1	Integrated box	m3.1_c1	Material	Aluminium
		m3.1_c2	Height	[200,1500] mm
		m3.1_c3	Width	[200,1000] mm
m3.2	Discrete box	m3.2_c1	Material	Icon
		m3.2_c2	Width	[200,1000] mm
m3.2.1	Cover	m3.2.1_c1	Height	[100,800] mm
m3.2.2	Base	m3.2.2_c1	Height	[100,800] mm
m4.2	Temperature control system 1	m4.2_c1	Control ability	Middle
m4.3	Temperature control system 2	m4.3_c1	Control ability	High

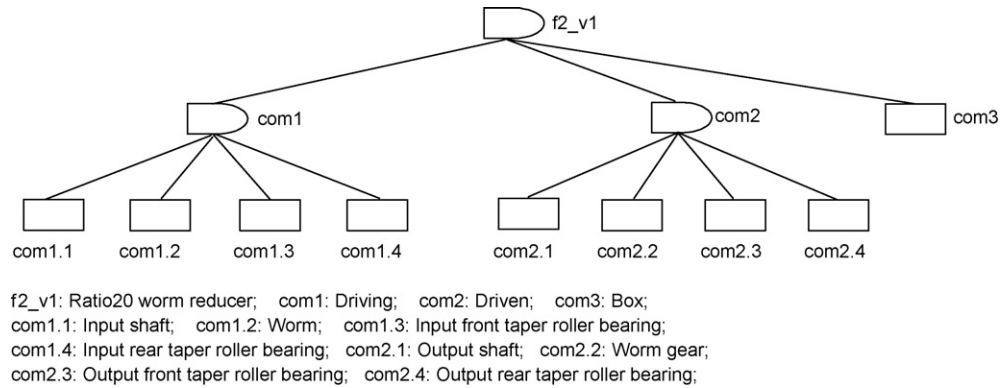


Fig. 6. Composition of a worm reducer variant.

configuration elements within their parental **module** element. For example, **configuration** elements between modules m1, m2, m3 and m4 are defined within their parental **module** element which is the **platform** element. Similarly, **configuration** elements between modules m1.1, m1.2, m1.3 and m1.4 are defined within their parental **module** element—m1.

The characteristics of the platform are defined by **property** elements within the corresponding element. In the worm reducer platform, some properties are customizable. They are either modelled as the sub-element of the **platform** element or the **module** element. Within the available region of the customizable properties, customers can specify the exact values for a certain characteristic. For example, the *ratio* **property** is a customizable characteristic. Customers can specify its values within bounds between $lbound = 10$ and $ubound = 50$. Some properties of a module have common values and some may be variables whose values will be instantiated during the customization stage. For common properties, their values have been given in the platform. They are kept the same from one variant to another in the family. For example, the *Material* **property** of the *Input shaft* **module** in the platform equals 45steel. All the worm reducers derived from the platform inherit the value of this property. The materials of their input shaft are all 45steel. For variable properties, the ranges of the possible values are also defined by the upper and lower bounds if they are continuous, or alternative values if they are discrete. For example, the *Modulus* **property** of the *Worm gear* **module**

element takes an integer value between $lbound = 1$ and $ubound = 25$. Its *width* **property** has been standardized into three levels of 15, 20 and 25 mm using the three **value** elements within the *width* **property** element.

4.2. ppXML variant element/document

A product variant is an instance of the associated platform. The composition, configuration and characteristics of the transmission box instance (variant) are shown in Figs. 6 and 7 and Table 2, respectively. There are significant similarities between the definitions of the platform and variants. The **component** element of the **variant** element is comparable to the **module** element of the **platform** element. The configuration relationships between **component** elements of the **reducer** variant are defined in the same way as those of the **reducer** platform. The hierarchical relationships of the variant composition are also implied by the nesting relationships between the **component** elements. Because the composition of a product variant is fixed at a point, there is no modular node (component element) with optional components. That is to say the *OR* **component** element will never ever appear in a variant.

The **component** and **variant** elements defined for the reducer variant have one or multiple **property** elements. All **property** elements defined for a variant have only point values, not value ranges for properties of platform modules. In other words, the **property** element defined for the **variant** element is

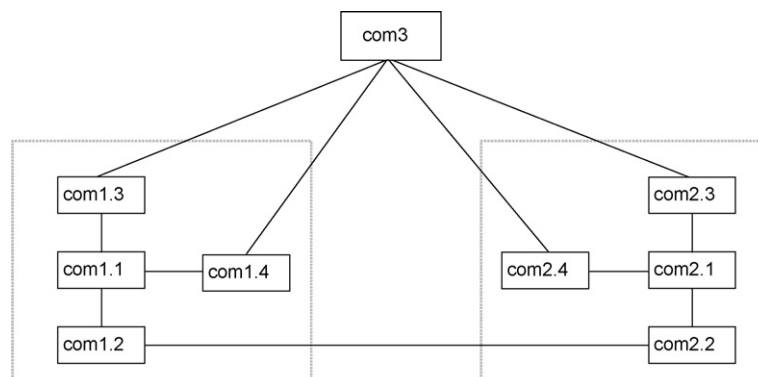


Fig. 7. Configuration of a worm reducer variant.

Table 2
Characteristics of a worm reducer variant

Component		Property	
id	Derived module	Derived property	Value
f2_v1	m1	f2_p1_c1	20
com1.1	m1.1.1	m1.1.1_c1	45steel
		m1.1.1_c2	50 mm
com1.2	m1.2	m1.2_c1	8 mm
		m1.2_c2	2
		m1.2_c3	80 mm
		m1.2_c4	20 mm
com1.3	m1.3	m1.3_c1	30211
com1.4	m1.4	m1.4_c1	30211
com2.1	m2.1	m2.1_c1	45steel
		m2.1_c2	60 mm
com2.2	m2.2	m2.2_c1	8 mm
		m2.2_c2	40
		m2.2_c3	320 mm
com2.3	m2.3	m2.3_c1	30206
com2.4	m2.4	m2.4_c1	30206
com3	m3.1	m3_c1	100 mm
		m3_c2	100 mm
		m3_c3	L
		m3.1_c1	Aluminium
		m3.1_c2	400 mm
		m3.1_c3	500 mm

slightly different from the **property** element defined for the **platform** element, although the same keyword is used for them. For example, the *Modulus* **property** of the *Worm gear component* defined for the reducer variant is derived from and therefore associated with the *Modulus* **property** of the

Worm gear module defined for the reducer platform. The former is set to a specific value of 5 in the range of 1–25 as defined by the latter.

5. Extending ppXML for mass customization grid

A computational grid called mcGrid has been proposed for mass customization and agile platform product development, as shown in Fig. 8. There are two groups of users as shown at the top of the figure. The first group of users includes the Web services providers who are responsible for providing PPD/MC methods and tools in the form of web services. The other group mainly includes web service consumers, e.g. product platform designers, product designers and customers who use Web services for solving specific product development problems. mcGrid provides facilities grouped under mcPortal and prdUDDI for these two user groups, respectively. The component on the left-bottom of the figure is ppXML—a special language devised to support the above two groups of facilities. The box on the right-bottom of the figure is a set of remote PPD/MC web services distributed and hosted on the Internet.

5.1. prdUDDI: special-purpose UDDI registry for PPD and MC

mcGrid is supported by a special-purpose Universal description, discovery and integration (UDDI) registry called prdUDDI. It provides a standard process to publish and discover, programmatically or through a graphical user interface (GUI), information about web services specialised in PPD and MC. At present, prdUDDI is implemented under Version 2.0 of the UDDI (http://uddi.org/pubs/uddi_v3.htm)

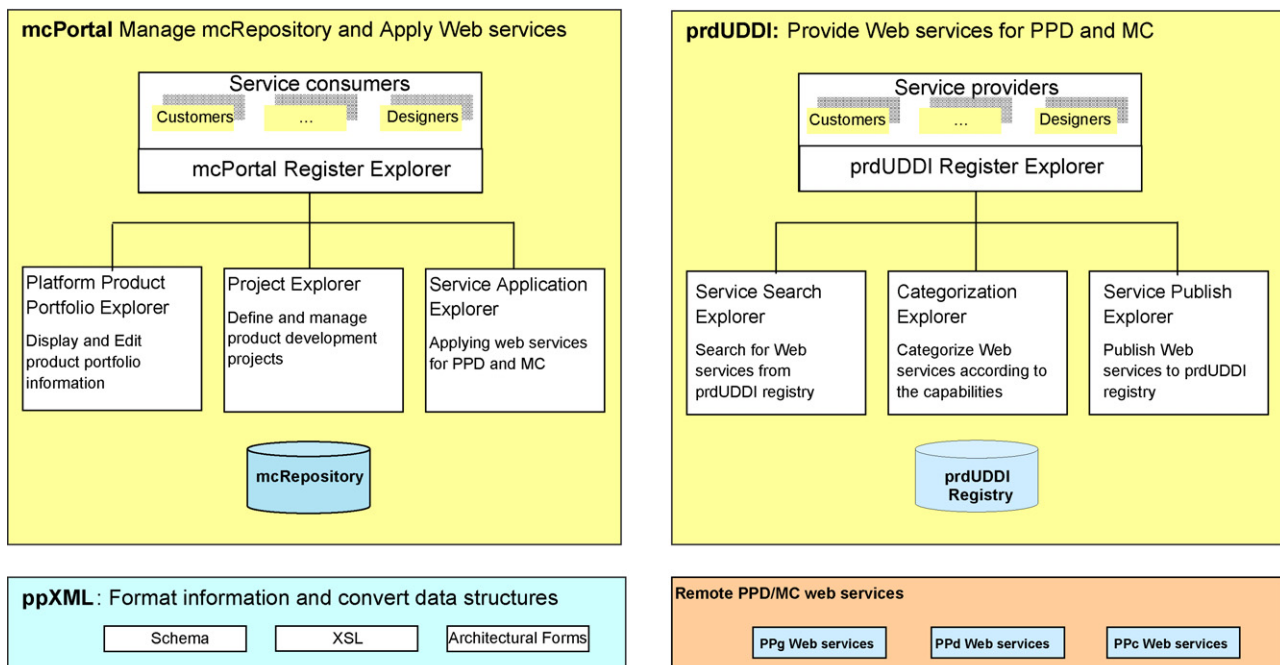


Fig. 8. Overview of mcGrid components.

specification for Web services, an industry-wide effort to bring a common standard for business-to-business (B2B) integration. prdUDDI consists of the following key components:

- prdUDDI (Business) Registry provides a highly XML-centric UDDI data model for encapsulating four types of information about web services, namely businessEntity, businessService, bindingTemplate and tModel.
- prdUDDI Register Explorer is a web-based GUI for user registration, during which information about businessEntity is captured and registered.
- Service Publish Explorer is a web-based GUI providing a set of facilities for defining and submitting the other three types (businessService, bindingTemplate and tModel) of information.
- Service Search Explorer is a web-based GUI for users to discover web services published at this prdUDDI most suitable for an application.
- Categorization Explorer is a web-based GUI which incorporates the valid values of several categorization schemes. It facilitates users to describe capabilities of PPD/MC web services.
- Remote web services are registered with, but do not belong to, the prdUDDI. Instead, they are hosted remotely by their providers.

5.2. *mcPortal: a web portal for applying web services for PPD and MC*

mcGrid provides this mcPortal group of facilities for the consumers of PPD and MC Web services. mcPortal consists of the repository and portfolio explorer components. mcRepository is a database for maintaining information and knowledge about users, projects, enterprise product strategies, and product designs. The data are structured according to ppXML constructs. mcRepository provides the centralized database to contain all the data and information related to PPD/MC projects in mcPortal. All inputs (design requirements) and outputs (product designs), together with resources (including product portfolios, families/models, variants, and platforms) used for PPD/MC are maintained at this mcRepository. In our study, a relational database is used for deploying this mcRepository. Data in the mcRepository are structured according to standard constructs devised in ppXML.

Platform Product Portfolio Explorer provides a set of GUI facilities for users to edit the contents of the ppXML documents and manipulate the contents of mcRepository conveniently by adding, deleting, and modifying the compositions, configurations, characteristics and constraints of the platform product. ppXML definition data of the product platform, variants and projects are stored in the mcRepository as the backend database, and displayed with the Product Portfolio Explorer as the frontend user interface. With the facilities provided in the Portfolio Explorer, ppXML document is automatically created and edited when the user conducting editing activities with the explorer's functions.

6. Concluding discussions

This paper has presented a new language called ppXML for defining platform products and their development projects. The original emphasis was in generalizing and simplifying the data model for wider distribution, easy access, web enabling the data for other applications and keeping it in public domain for maintenance and updating. The outcome of ppXML schema development will provide the platform product design and development software developers to work with one standard format for describing data and develop Web services.

Adopting XML for platform product data representation would enable different Web services involved in product design and development to work with a single data format and allow other applications to make use of this data or even produce part of this data. ppXML includes a set of schemas to standardize tag terminologies for platform product data such that web solutions interested in interoperability can provide an API conforming to this modelling language. Therefore, web solutions can be incrementally developed, deployed and registered on prdUDDI, the UDDI registry provided by the prototype system. On the whole, ppdPortal will enable and promote the industrial applications of the platform product development approach through appropriate decision support systems in the form of e-business solutions (EBS).

In comparison with previous efforts, ppXML exhibits a number of important features. Firstly, ppXML can be used not only for defining products, but also product platforms, for mass customization. Other product definition languages are not devised for this application. Secondly, ppXML is suitable for product planning using the platform approach at the strategic level while other languages are mainly dedicated to product definition at the detailed level. Finally, ppXML offers a wider perspective of extension and application in combination with WSDL (<http://www.w3.org/TR/2006/CR-wsdl20-primer-20060106/>) and SOAP (<http://www.w3.org/TR/2003/REC-soap12-part0-20030624/>) messages for web services and computational grid specifically developed for platform product development and mass customization.

As a product platform definition language, ppXML is able to describe typical platform strategies based on the generic bill of materials. For example, platform commonality is defined by the fixed modules and their combinations; platform modularity is reflected by the “Or” type platform modules and their options; platform scalability is defined by scalable module characteristics or properties; and platform postponement is reflected by the high-level locations of the “Or” type modules and low-level locations of the “And” type modules in the module nestings.

The future work on ppXML should be directed towards its uses in various decision support systems for different applications in platform product development. This is particularly important because data requirements of these systems vary widely with the underlying methodologies. For example, all product variants must be defined in ppXML as an input file to a decision support system for developing the product platform from the existing product variants. In contrast, customer requirements of representative market segments must be

defined in ppXML as an input document if the decision support system develops the product platform from the market domain. Fortunately, the XML technology provides the necessary flexibility and extensibility for these purposes.

References

- [1] B.J. Pine II, Mass Customization: The New Frontier in Business Competition, Harvard Business School Press, Boston, MA, 1993.
- [2] D.M. Anderson, B.J. Pine II, Agile product development for mass customization, Irwin Publishers, Chicago, IL, 1997.
- [3] M.M. Tseng, J. Jiao, Design for mass customization by developing product family architecture, ASME Design Engineering Technical Conferences—Design Theory and Methodology, Paper No. DETC98/DTM-5717, Atlanta, GA, ASME, 1998.
- [4] M.H. Meyer, A.P. Lehnerd, The Power of Product Platforms: Building Value and Cost Leadership, Free Press, New York, NY, 1997.
- [5] K. Ulrich, The role of product architecture in the manufacturing firm, Research Policy 24 (3) (1995) 419–440.
- [6] D. Robertson, K. Ulrich, Planning product platforms, Sloan Management Review 39 (4) (1998) 19–31.
- [7] D.E. Whitney, Nippondenso co. Ltd: a case study of strategic product design, Research in Engineering Design 5 (1) (1993) 1–20.
- [8] K.T. Ulrich, S.D. Eppinger, Product Design and Development, second ed., McGraw-Hill, Inc, New York, NY, 2000.
- [9] J.S. Spira, Mass customization through training at lutron electronics, Planning Review 21 (4) (1993) 23–24.
- [10] E. Schonfeld, The customized, digitized, have-it-your way economy, Fortune 138 (6) (1998) 114–124.
- [11] S.W. Sanderson, M. Uzumeri, Managing Product Families, Irwin, Chicago, IL, 1997.
- [12] S. Kota, K. Sethuraman, Managing variety in product families through design for commonality, in: Proceedings of 1998 ASME DETC, DETC98/DTM-5651, Atlanta, 1998.
- [13] F. van der Linden, J.G. Wijnstra, Platform engineering for the medical domain, in: F. van der Linden (Ed.), Software Product-Family Engineering, Fourth International Workshop PFE 2001, Bilbao, Spain, (2001), pp. 224–237.
- [14] Dell, 2000, Dell Computers, from <http://www.dell.com/>.
- [15] Cannondale, 2000, Connadale bicycles, from <http://www.cannondale.com/>.
- [16] Haworth, 2000, Haworth furniture, from <http://www.haworth-furn.com/>.
- [17] IDtown, 2000, Customized watches, from <http://www.idtwon.com/>.
- [18] Customatix, 2000, Customatix shoes, from <http://www.Customatix.com/>.
- [19] Eyeplanet, 2000, Eyeplanet optics, from <http://www.eyeplanet.com/>.
- [20] Volvo, 2000, Design your own Volvo, from <http://www.volvocars.com/>.
- [21] F.L. Krause, F. Kimura, T. Kjellberg, S.C.Y. Lu, Product modelling, Annals of the CIRP 42 (1) (1993) 695–704.
- [22] J.S. Gero, U. Kannengiesser, The situated function-behaviour-structure framework, in: J.S. Gero (Ed.), Artificial Intelligence in Design'02, Kluwer Academic Publishers, Dordrecht, 2002, pp. 89–104.
- [23] D. Xue, S. Yadav, D.H. Norrie, Knowledge base and database representation for intelligent concurrent design, Computer-Aided Design 31 (1999) 131–145.
- [24] G.Q. Huang, S.W. Lee, K.L. Mak, Web-based product and process data modelling in concurrent 'Design for X', International Journal of Robotics and Computer Integrated Manufacturing 15 (1) (1999) 53–63.
- [25] P.H. Gu, S. Sasale, Product modularization for life cycle engineering, Robotics and Computer Integrated Manufacturing 16 (5) (1999) 387–402.
- [26] T. Mannisto, A conceptual modelling approach to product families and their evolution, Doctorial dissertation, Helsinki University of Technology, Helsinki, Finland, 2000.
- [27] F. Erens, K. Verhulst, Architectures for product families, Computers in Industry 33 (2–3) (1997) 165–178.
- [28] E. van Veen, Modeling Product Structures by Generic Bills-of-Materials, Elsevier Science Publishers, Amsterdam, 1992.
- [29] A. Winter, B. Kullbach, V. Riediger, An overview of the GXL graph exchange language, Paper presented at Software Visualisation International Seminar, Dagstuhl, Saarland, Germany, 2001.
- [30] T. Mannisto, H. Peltonen, T. Soininen, R. Sulonen, Multiple abstraction levels in modelling product structures, Data & Knowledge Engineering 36 (2001) 55–78.
- [31] M. Tseng, Modeling Product Family Architecture For Mass Customization, Hong Kong Government Research Grants Council—CERG, Awarded to Professor Mitchell Tseng at Advanced Manufacturing Institute, Department of Industrial Engineering and Engineering Management, Hong Kong University of Science and Technology (HKUST), 1999.
- [32] J. Jiao, M.M. Tseng, Understanding product family for mass customization by developing commonality indices, Journal of Engineering Design 11 (3) (2000) 225–243.
- [33] X. Du, Architecture of product family for mass customisation, Doctorial dissertation, Hong Kong University of Science and Technology, Hong Kong, China, 2000.
- [34] E.M. Dashofy, A. van der Hoek, Representing product family architecture in an extensible architecture description language, in: Proceedings of the International Workshop on Product Family Engineering (PFE-4), Bilbao, Spain, 2001.



Dr. George Huang is associate professor in the Department of Industrial and Manufacturing Systems Engineering, the University of Hong Kong. He obtained his B.Eng. degree in mechanical engineering from Southeast University (China), and Ph.D. degree in mechanical engineering from the University of Wales Cardiff (UK). His main research areas include collaborative product commerce, digital manufacturing. He has published extensively in these topics, including two monographs entitled *Cooperating Expert Systems in Mechanical Design and Internet Applications in Product Design and Manufacturing* respectively, and an edited reference book entitled *Design for X: Concurrent Engineering Imperatives*. Dr. Huang is a Chartered Engineer, and a member of IEE, ASME, IIE, and HKIE.



Miss Li Li is a Ph.D. researcher in the Department of Industrial and Manufacturing Systems Engineering, the University of Hong Kong. She received her B.E. and M.S. degrees in mechanical engineering from Dalian University of Technology (China), in 2000 and 2003, respectively. Her research interests include mass customization, product design, product platform/family optimization, and evolutionary algorithms.

Dr. Xin Chen is professor of electromechanical engineering and Vice President of Guangdong University of Technology. He obtained bachelor degree from Chang Sha Railway Institute, master degree from Harbin Institute of Technology and doctor degree from Hua Zhong University of Science and Technology. He was visiting professor in Nottingham University in 2002. Over the years, professor Chen has been in charge of a number of major projects of National Natural Science Foundation of China and National 863 High-Tech Research and Development Program of China.